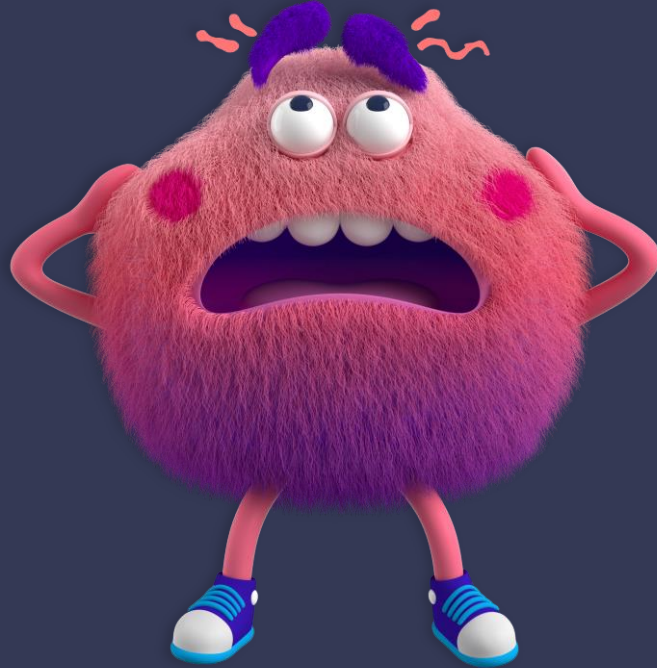


Hash Table



چرا به ساختار داده هش (Hash) نیاز داریم؟

با رشد تصاعدی داده‌ها در دنیای فناوری، ذخیره‌سازی، بازیابی و پردازش کارآمد اطلاعات به یکی از چالش‌های اصلی علوم کامپیوتر تبدیل شده است. در برنامه‌نویسی روزمره، یکی از رایج‌ترین ساختارهای داده برای ذخیره‌سازی، «آرایه» (Array) است.

اما سوال اینجاست: اگر ساختار داده‌ای مانند آرایه وجود دارد، چه نیازی به ابداع جداول هش (Hash Tables) بود؟

پاسخ این سوال در یک کلمه خلاصه می‌شود: «کارایی» (Efficiency) در جستجو.

محدودیت‌های ساختار داده آرایه (Array)

آرایه‌ها برای دسترسی به داده‌ها از طریق «اندیس» (Index) بسیار عالی هستند و زمان دسترسی به یک اندیس مشخص در آن‌ها **ثابت** و برابر با $O(1)$ است. اما مشکل زمانی آغاز می‌شود که ما اندیس داده را نمی‌دانیم و می‌خواهیم بر اساس «محتوا یا مقدار» جستجو کنیم. در اینجا **آرایه** به شدت **ناکارآمد** می‌شود:

۱. در **آرایه‌های مرتب نشده (Unsorted Arrays)**: برای پیدا کردن یک داده خاص، باید عناصر را یکی‌یکی از ابتدا تا انتها بررسی کنیم (**Linear Search**). پیچیدگی زمانی این کار $O(n)$ است.

۲. در **آرایه‌های مرتب شده (Sorted Arrays)**: اگرچه می‌توانیم از الگوریتم‌های سریع‌تری مانند جستجوی باینری (Binary Search) استفاده کنیم که پیچیدگی زمانی را به $O(\log n)$ کاهش می‌دهد، اما برای مجموعه داده‌های بسیار بزرگ (مثلاً میلیون‌ها رکورد)، همین مقدار زمان نیز پردازش سنگینی می‌طلبد. به علاوه، درج (Insertion) یک داده جدید در **آرایه مرتب‌شده** نیازمند **جابجایی سایر عناصر** است که زمان $O(n)$ را به سیستم تحمیل می‌کند.

ورود جدول هش (Hash Table) به میدان

به دلیل محدودیت‌های آرایه، مهندسان نرم‌افزار به دنبال ساختار داده‌ای بودند که بتواند عملیات **جستجو، درج و حذف** داده‌ها را بر اساس «کلید» در زمان **ثابت**، یعنی $O(1)$ انجام دهد.

اینجا بود که ساختار داده **هش (Hash Table)** معرفی شد. **Hash Table** با استفاده از یک مکانیزم هوشمندانه (توابع هش) نیاز به **جستجوی خطی یا باینری** را از بین می‌برد.

تفاوت اصلی در نحوه دسترسی:

در حالی که در آرایه شما مجبورید برای یافتن داده سوال بپرسید: “آیا این داده در خانه اول است؟ دوم؟ یا...”، در **Hash Table**، شما «کلید» را به تابع هش می‌دهید و این تابع بلافاصله با یک محاسبه ریاضی در زمان $O(1)$ به شما می‌گوید: “داده شما دقیقاً در اندیس شماره X در حافظه قرار دارد.”

مقایسه نهایی و فنی :

✓ جستجو (Search): در آرایه بین $O(n)$ تا $O(\log n)$ زمان می‌برد، اما در Hash Table به طور میانگین $O(1)$ است.

✓ درج (Insertion): در آرایه (در صورت نیاز به جابجایی عناصر یا پیدا کردن جایگاه) $O(n)$ زمان می‌برد، اما در Hash Table (بدون در نظر گرفتن زمان تغییر اندازه جدول) $O(1)$ است.

✓ حذف (Deletion): مانند درج، در آرایه $O(n)$ و در جدول هش $O(1)$ است.

به همین دلیل، هرکجا که سرعت بالا در جستجو و تطبیق داده‌ها (مانند سیستم‌های کش، دیتابیس‌ها و دیکشنری‌ها) حیاتی باشد، ساختار داده هش جایگزین بلامنازع آرایه‌ها می‌شود.

همانطور که بررسی کردیم، جداول هش زمان جستجو را در مقایسه با آرایه‌های سنتی به $O(1)$ کاهش می‌دهند. در واقع، طراحی Hash Table تجلی کامل مفهوم «موازنه فضا-زمان» (Space-Time Tradeoff) در مهندسی نرم‌افزار است.

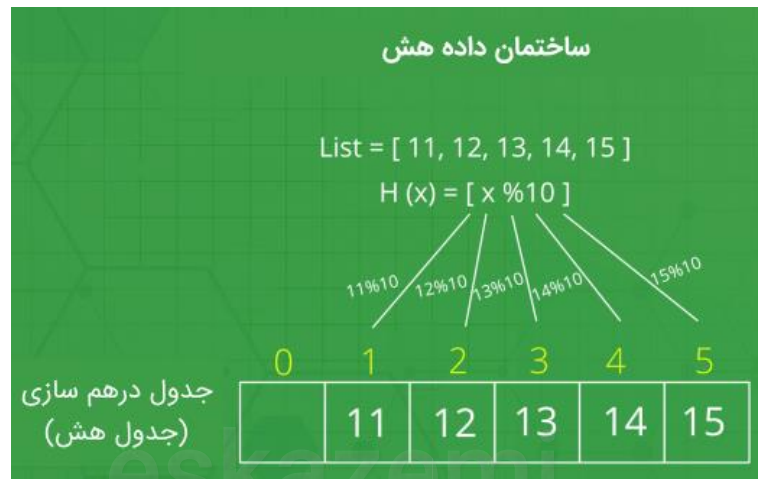
این ساختار یک راهکار میانه بین دو حالت افراطی است:

- ✓ اگر زمان بی‌نهایت داشتیم: نیازی به جدول هش نبود! می‌توانستیم تمام داده‌ها را در یک ساختار ساده (مانند یک آرایه یا لیست پیوندی در یک اندیس واحد) ذخیره کنیم و هر بار با جستجوی خطی $O(n)$ یا جستجوی باینری $O(\log n)$ داده‌ها را پیدا کنیم. این کار کمترین فضای ممکن را اشغال می‌کند، اما زمان زیادی می‌طلبد.
- ✓ اگر فضای حافظه بی‌نهایت داشتیم: باز هم نیازی به توابع هش پیچیده نبود! می‌توانستیم خود «کلید» را مستقیماً به عنوان اندیس آرایه در نظر بگیریم (که به آن Direct Addressing می‌گویند). در این حالت زمان دسترسی مطلقاً $O(1)$ بود و هیچ برخوردی (Collision) رخ نمی‌داد، اما به تعداد بی‌نهایت (یا به اندازه تمام کلیدهای ممکن) به خانه‌های حافظه نیاز داشتیم که در دنیای واقعی غیرممکن است.

از آنجا که ما در دنیای واقعی نه زمان بی‌نهایت داریم و نه حافظه بی‌نهایت، Hash Table به عنوان یک راهکار بهینه وارد عمل می‌شود. این ساختار با استفاده از تابع هش، فضای کمتری نسبت به حالت دوم اشغال می‌کند و سرعت بسیار بالاتری نسبت به حالت اول ارائه می‌دهد. البته بهای این موازنه، پذیرش پدیده‌ای به نام «تصادم یا برخورد» (Collision) است که باید مدیریت شود.

اکنون که متوجه شدیم برای دستیابی
به پیچیدگی زمانی $O(1)$ نیازمند
مکانیزم جدیدی هستیم، باید به
سراغ مفهوم هشینگ (Hashing)
برویم.

Hashing



مفهوم هشینگ (Hashing) چیست؟

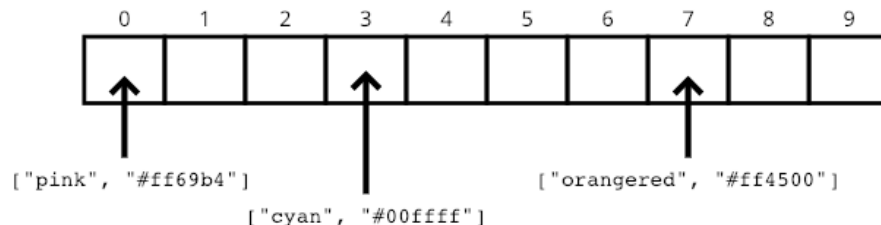
در علوم کامپیوتر، هشینگ به فرآیند تبدیل داده‌های ورودی (با هر فرمت و طول متغیر) به یک خروجی ریاضی **قطعی با طول ثابت** گفته می‌شود. این عملیات توسط الگوریتم‌های خاصی به نام **توابع هش (Hash Functions)** انجام می‌پذیرد. خروجی تولید شده، یک شناسه فشرده است که به آن «**کد هش (Hash Code)**» یا «**مقدار هش (Hash Value)**» می‌گویند.

نقش هشینگ در ساختار داده:

رسالت اصلی تکنیک هشینگ در ذخیره‌سازی داده‌ها، ایجاد یک سیستم «**آدرس‌دهی مستقیم**» است. تابع هش با دریافت یک ورودی، مستقیماً **اندیس (Index)** یا **آدرس دقیق حافظه** را برای ذخیره‌سازی یا بازیابی آن آیتم محاسبه می‌کند. به زبان ساده‌تر، هشینگ همان موتور محرکی است که نیاز به جستجوی ترتیبی در آرایه‌ها را حذف کرده و **دسترسی به داده‌ها را در زمان ثابت** ممکن می‌سازد.

Hash Table

HASHING CONCEPTUALLY



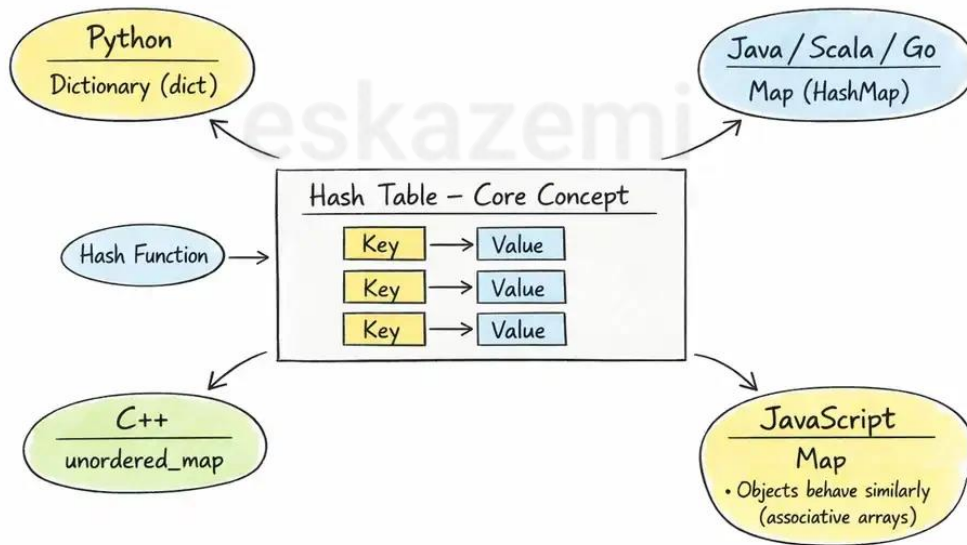
تجلی هشینگ در Hash Table ها :

الگوریتم‌های هشینگ در قالب ساختار داده‌ای به نام **Hash Table** پیاده‌سازی می‌شوند. در این معماری از **الگو** و جفت‌های «**کلید-مقدار**» (**Key-Value**) استفاده می‌شود؛ جایی که «کلید» برای محاسبه اندیس به تابع هش پاس داده می‌شود و «مقدار» (داده اصلی) در آن اندیس محاسبه‌شده در حافظه قرار می‌گیرد.

پیاده‌سازی در زبان‌های برنامه‌نویسی:

به دلیل کارایی بی‌نظیر و سرعت بالا در عملیات درج، حذف و جستجو، ساختار **Hash Table** در هسته مرکزی (Standard Library) اکثر زبان‌های برنامه‌نویسی مدرن به شکل بومی پیاده‌سازی شده است. اگرچه مکانیزم زیرین همه آن‌ها یکسان است، اما در زبان‌های مختلف با نام‌های متفاوتی شناخته می‌شوند:

Hash Table Implementations in Modern Programming Languages



All share the same core Hash Table mechanism — optimized for fast Insert, Delete, and Lookup operations.

Components of Hashing

۱. کلید (Key):

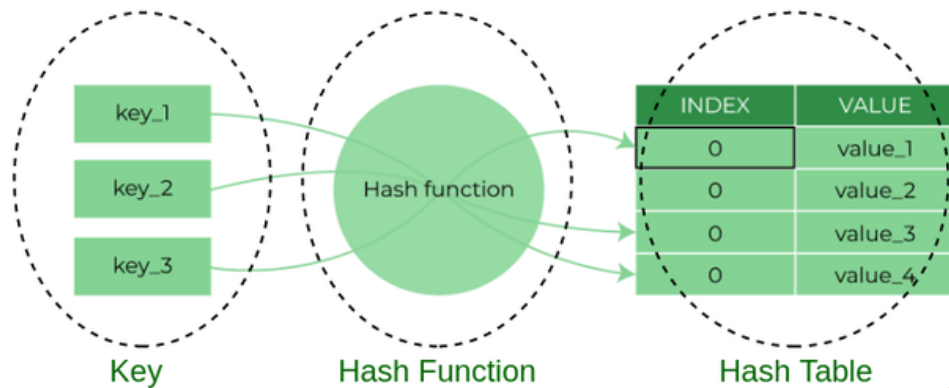
کلید یک شناسه ورودی (مانند یک رشته متنی، عدد یا هر ساختار داده‌ای پشتیبانی شده) است که برای **ذخیره** و **بازیابی اطلاعات** استفاده می‌شود. این کلید به عنوان **ورودی** به **تابع هش** پاس داده می‌شود تا جایگاه دقیق داده‌ها در **حافظه** مشخص شود.

۲. تابع هش (Hash Function):

تابع هش یک **الگوریتم ریاضی** است که داده‌های ورودی (با هر اندازه و طولی) را دریافت کرده و آن‌ها را به یک مقدار خروجی **قطعی** و با **طول ثابت** تبدیل می‌کند (که به آن Hash Value یا Hash Code می‌گویند). در ساختار Hash Table، وظیفه اصلی این تابع این است که «کلید» را دریافت کند و از روی آن، یک **اندیس (Index)** عددی محاسبه کند. این اندیس نشان می‌دهد که داده‌های مرتبط با آن کلید باید در کجای آرایه (Hash Table) ذخیره شوند.

نکته فنی بسیار مهمی که در اینجا باید به آن توجه کرد، **تغییر حجم داده‌ها (تعداد بایت‌ها) در خروجی Hash** است. از آنجا که **خروجی همیشه طول ثابتی دارد**، Hash Function می‌تواند تعداد بایت‌های داده را کاهش یا افزایش دهد.

- ✓ **کاهش حجم بایت‌ها:** اگر ورودی شما یک فایل ویدیویی ۱ گیگابایتی باشد، خروجی تابع هش مثلاً در الگوریتم SHA-256 تنها ۲۵۶ بیت (۳۲ بایت) خواهد بود.
- ✓ **افزایش حجم بایت‌ها:** اگر ورودی شما تنها یک حرف مانند a (یک بایت) باشد، خروجی تابع هش از همان الگوریتم SHA-256 استفاده کنیم همچنان همان ۲۵۶ بیت (۳۲ بایت) خواهد بود.



جدول هش (Hash Table) :

جدول هش یک ساختار داده است که برای پیاده‌سازی دیتاتایپ‌های «کلید-مقدار» (Key-Value) مانند دیکشنری‌ها استفاده می‌شود. این ساختار در هسته خود از یک آرایه برای ذخیره داده‌ها استفاده می‌کند.

در یک آرایه معمولی، داده‌ها بر اساس اعداد ترتیبی (اندیس ۰، ۱، ۲ و غیره) ذخیره می‌شوند. اما در **Hash Table**، جایگاه (اندیس) ذخیره‌سازی داده‌ها از قبل مشخص نیست. روال کار به این شکل است:

- ✓ شما یک **کلید** و یک **مقدار** (داده) دارید.
- ✓ تابع هش، کلید را می‌گیرد و یک پردازش ریاضی روی آن انجام می‌دهد تا یک عدد (اندیس) تولید کند.
- ✓ سپس جدول هش، داده شما را دقیقاً در همان اندیس محاسبه شده در آرایه ذخیره می‌کند.
- ✓ در زمان جستجو نیز با دادن کلید به تابع هش، اندیس فوراً محاسبه شده و داده‌ها بدون نیاز به جستجوی ترتیبی پیدا می‌شوند. به این ویژگی «دسترسی مستقیم از طریق نگاشت» می‌گویند.

نکته : ساختار Hash Table دارای ویژگی **ضریب بار** یا **Load Factor** است Hash Table وقتی ضریب بار $(\alpha = \frac{n}{m})$ از حدی بالاتر رفت، تصمیم به **Resize** و **بزرگ کردن فضای آرایه** می‌گیرد).

هشینگ (Hashing) در عمل چگونه کار می‌کند؟ (یک مثال کاربردی)

برای درک بهتر مکانیزم هشینگ، بیایید یک سناریوی عملی را بررسی کنیم. فرض کنید مجموعه‌ای از رشته‌ها شامل `"ab", "cd", "efg"` داریم و می‌خواهیم آن‌ها را در یک **ساختار داده** ذخیره کنیم.

هدف معماری ما در اینجا، دستیابی به سرعت **$O(1)$** برای عملیات جستجو، درج و به‌روزرسانی است و **حفظ «ترتیب» (Order)** داده‌ها برای ما **اهمیتی ندارد**. در این مثال خاص (پیاده‌سازی یک Hash Set)، خود رشته‌ها هم نقش «کلید» (**Key**) را برای محاسبه آدرس بازی می‌کنند و هم نقش «مقدار» (**Value**) را برای ذخیره شدن در حافظه دارند.

فرآیند ذخیره‌سازی این داده‌ها در جدول هش طی مراحل زیر انجام می‌شود:

مرحله اول: تبدیل کلید به یک عدد صحیح (تولید Hash Code)

تابع هش در قدم اول باید داده‌های متنی ما را به یک مقدار عددی تبدیل کند. برای سادگی این مثال، فرض می‌کنیم یک نگاشت (Mapping) ساده برای حروف الفبا تعریف کرده‌ایم؛ به این صورت که $a = 1$, $b = 2$, $c = 3$ و الی آخر (نکته فنی: در سیستم‌های واقعی معمولاً از مقادیر استاندارد *ASCII* یا *Unicode* کاراکترها استفاده می‌شود).

اکنون الگوریتم ما ارزش عددی هر رشته را با جمع کردن مقادیر کاراکترهای آن محاسبه می‌کند:

✓ رشته `ab` : ارزش عددی برابر است با $1 + 2 = 3$

✓ رشته `cd` : ارزش عددی برابر است با $3 + 4 = 7$

✓ رشته `efg` : ارزش عددی برابر است با $5 + 6 + 7 = 18$

Hash Table

مرحله دوم: نگاشت عدد به اندیس آرایه (Compression Function)

اعداد به دست آمده (3,7,18) را نمی‌توان مستقیماً به عنوان جایگاه حافظه استفاده کرد، زیرا ممکن است از ظرفیت آرایه ما فراتر باشند. فرض کنید ما یک جدول هش (آرایه) با ظرفیت ۷ خانه داریم (همانطور که در تصویر مشخص است، اندیس‌ها از 0 تا 6 شماره‌گذاری شده‌اند).

برای اینکه مقادیر تولید شده در محدوده ظرفیت جدول ما قرار بگیرند، تابع هش از عملگر ریاضی «باقی‌مانده تقسیم» (Mod یا Modulo) استفاده می‌کند. فرمول محاسبه اندیس نهایی به این شکل است:

$$\text{Index} = \text{Hash Code} \pmod{\text{Table Size}}$$

مرحله سوم: محاسبه اندیس نهایی و ذخیره‌سازی

حالا مقادیر تولید شده در مرحله اول را در فرمول بالا قرار می‌دهیم تا موقعیت دقیق هر رشته در جدول (آرایه ۷ خانه‌ای) مشخص شود:

✓ مکان رشته ab: باقی‌مانده تقسیم 3 بر 7 می‌شود $3 \pmod{7} = 3$ → اندیس 3

✓ مکان رشته cd: باقی‌مانده تقسیم 7 بر 7 می‌شود $7 \pmod{7} = 0$ → اندیس 0

✓ مکان رشته efg: باقی‌مانده تقسیم 18 بر 7 می‌شود $18 \pmod{7} = 4$ → اندیس 4

با نگاه به تصویر جدول هش، دقیقاً همین معماری را مشاهده می‌کنید: داده cd در خانه 0، داده ab در خانه 3 و داده efg در خانه 4 بدون هیچ ترتیب خاصی ذخیره شده‌اند.

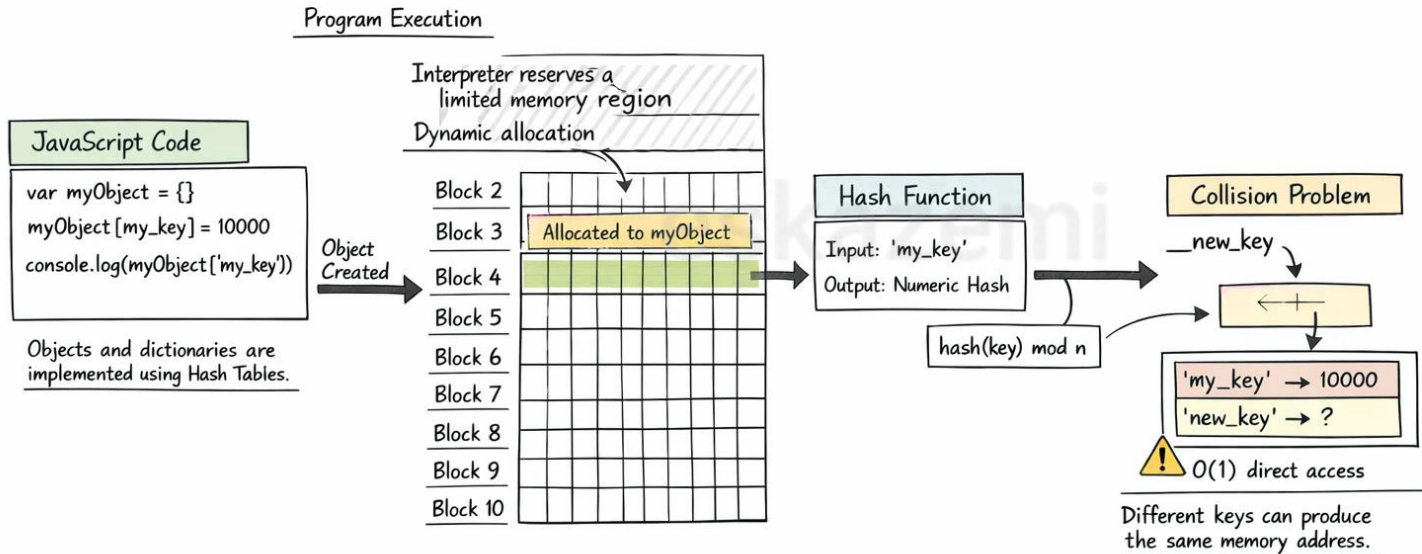
0	1	2	3	4	5	6
cd			ab	efg		

نتیجه‌گیری فنی از این مکانیزم

تکنیک بالا نشان می‌دهد که چگونه تابع هش ما را قادر می‌سازد مکان دقیق یک داده را فوراً محاسبه کنیم. اگر در آینده سیستم بخواهد بداند آیا رشته efg در جدول وجود دارد یا خیر، نیازی ندارد خانه‌های 0 تا 3 را جستجو کند؛ بلکه مستقیماً همان محاسبات $(5 + 6 + 7) \pmod{7}$ را انجام داده و یک‌راست به سراغ اندیس 4 می‌رود و داده را در زمان ثابت $O(1)$ پیدا می‌کند.

(نکته فنی تکمیلی: الگوریتمی که در این مثال استفاده کردیم، یعنی «جمع کاراکترها»، یک تابع هش بسیار ساده و ضعیف است. زیرا برای مثال، کلمات ab و ba هر دو خروجی یکسان (3) تولید می‌کنند و می‌خواهند در یک اندیس ذخیره شوند. در مهندسی نرم‌افزار به این پدیده «تصادم» (**Collision**) می‌گویند که توابع هش پیشرفته با الگوریتم‌های پیچیده‌تر و استفاده از روش‌هایی مانند Chaining آن را مدیریت می‌کنند).

How Hash Tables Work in RAM (Behind the Scenes)



نگاهی به پشت صحنه:
جداول هش در حافظه
(RAM) چگونه رفتار می‌کنند؟
برای درک بهتر اینکه چرا
توابع هش همیشه بی‌نقص
نیستند، بیایید نگاهی به
نحوه پیاده‌سازی آن‌ها در یک
زبان برنامه‌نویسی واقعی
مانند جاوا اسکریپت
بیندازیم.

Hash tables map keys to memory addresses using a hash function. Because memory space is limited but possible keys are unlimited, collisions are inevitable.

Hash Table

به قطعه کد زیر که یک شیء (Object) ساده را تعریف می‌کند توجه کنید:
در زبان‌هایی مانند جاوا اسکریپت یا پایتون، اشیاء (Objects) و دیکشنری‌ها (Dictionaries) در لایه زیرین خود دقیقاً همان **جداول هش** (**Hash Tables**) هستند. اما وقتی این سه خط کد اجرا می‌شود، در پشت صحنه (Background) سیستم و حافظه RAM چه اتفاقی می‌افتد؟

۱. تخصیص حافظه (Memory Allocation)

وقتی در خط اول متغیر myObject را تعریف می‌کنیم، سیستم عامل منطقاً کل فضای RAM را به این متغیر اختصاص نمی‌دهد. بلکه یک بلوک مشخص و محدود از حافظه را برای آن رزرو می‌کند.

در علم کامپیوتر، ما دو نوع تخصیص حافظه برای آرایه‌ها داریم:

• **آرایه‌های ایستا (Static Arrays):** در زبان‌هایی مانند C یا Java، شما باید از همان ابتدا سائز آرایه را مشخص کنید (مثلاً بگویید من یک آرایه با ظرفیت ۱۰۰ خانه می‌خواهم). سیستم دقیقاً همان مقدار فضا را به شما می‌دهد.

• **آرایه‌های پویا (Dynamic Arrays):** در زبان‌هایی مثل جاوا اسکریپت یا پایتون، نیازی به تعیین سائز نیست. سیستم ابتدا یک فضای کوچک (مثلاً ۸ خانه) را اختصاص می‌دهد و اگر پر شد، در پشت صحنه یک آرایه بزرگ‌تر می‌سازد و داده‌ها را به آن منتقل می‌کند.

برای درک بهتر، فرض کنید RAM سیستم ما کلاً شامل ۱۰۰۰ بلوک (قفسه) حافظه است. وقتی myObject ساخته می‌شود، مفسر جاوا اسکریپت مثلاً **بلوک‌های شماره ۲ تا ۱۰** را به این آبجکت اختصاص می‌دهد.

```
var myObject = {};  
myObject['my_key'] = 10000;  
console.log(myObject['my_key']);
```

Hash Table

تولید هش و نگاشت به حافظه (Hashing & Mapping)

در خط دوم کد (`10000myObject['my_key'] =`)، ما می‌خواهیم مقدار 10000 را با کلید `my_key` ذخیره کنیم. در اینجا موتور جاوا اسکریپت مراحل زیر را طی می‌کند:

1. ابتدا کلید متنی `'my_key'` را به تابع هش داخلی خود می‌دهد.

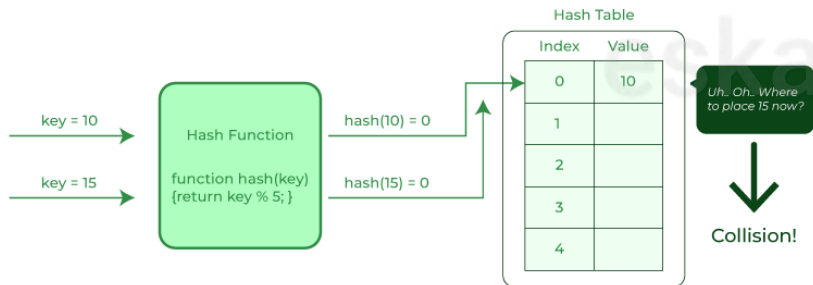
2. تابع هش یک کد عددی تولید می‌کند.

3. سپس با استفاده از عملگرهای ریاضی (مانند $\text{mod } n$) که در بخش‌های قبل بررسی کردیم، این کد هش را به یک آدرس دقیق در همان محدوده حافظه اختصاص یافته تبدیل می‌کند. به عبارت دیگر، تابع هش موظف است هر کلیدی که به آن داده می‌شود را به یک آدرس معتبر در بازه **۲ تا ۱۰ بلوک** تبدیل کند (مثلاً فرض کنید خروجی محاسبات برای `'my_key'` بلوک شماره ۷ شده است). در نهایت، عدد 10000 در آن قفسه از حافظه نوشته می‌شود.

در خط سوم نیز برای چاپ کردن (Log) داده، دقیقاً همین فرآیند تکرار می‌شود: کلید `'my_key'` هش شده، مستقیماً به بلوک شماره ۷ اشاره می‌کند و مقدار 10000 در زمان $O(1)$ از حافظه خوانده می‌شود.

```
var myObject = {};  
myObject['my_key'] = 10000;  
console.log(myObject['my_key']);
```

Collision in Hashing



سرآغاز یک مشکل: پدیده تصادم (Collision)

با درک مکانیزم بالا، به یک نتیجه‌گیری منطقی و البته چالش‌برانگیز می‌رسیم:

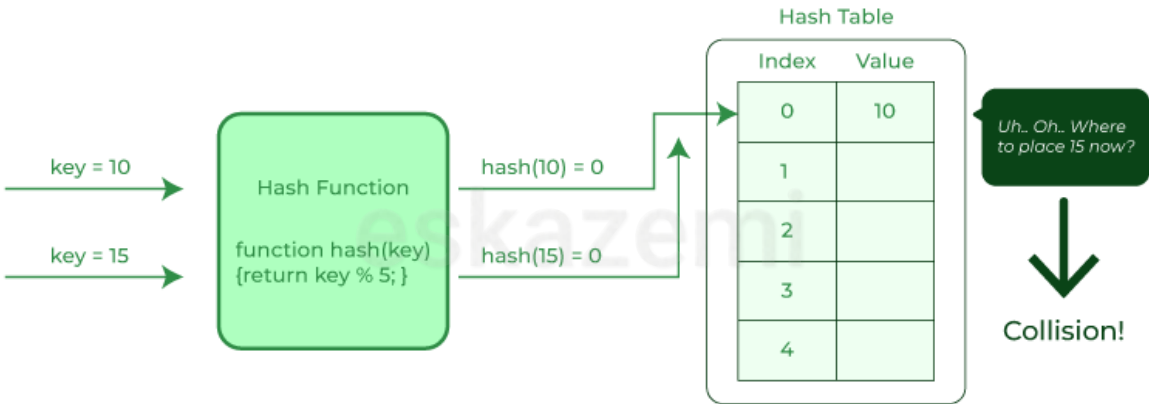
فضای تخصیص داده شده به متغیر ما محدود است (در مثال ما تنها ۹ بلوک، از ۲ تا ۱۰)، اما تعداد کلیدهایی که می‌توانیم به عنوان ورودی به تابع هش بدهیم بی‌نهایت است (ما می‌توانیم هر رشته متنی دلخواهی را به عنوان کلید استفاده کنیم).

طبق اصل لانه کبوتری در ریاضیات، زمانی که بخواهیم بی‌نهایت ورودی را در یک فضای محدود (مثلاً ۹ خانه) جا بدهیم، تابع هش ما دیر یا زود برای **دو کلید کاملاً متفاوت، آدرس تکراری و یکسانی** تولید خواهد کرد. برای مثال، ممکن است سیستم بخواهد کلید new_key را ذخیره کند، اما خروجی تابع هش برای این کلید جدید نیز **بلوک شماره ۷** باشد؛ همان جایی که قبلاً my_key در آن نشسته است! در مهندسی نرم‌افزار، به این اتفاق که دو داده مختلف بر سر یک خانه از حافظه با هم رقابت می‌کنند، **«تصادم یا برخورد» (Collision)** می‌گویند.

اما ساختارهای داده چگونه این تداخل را مدیریت می‌کنند تا داده‌های قبلی پاک نشوند؟ این دقیقاً همان موضوعی است که در بخش بعدی به تفصیل آن را بررسی خواهیم کرد.

Hash Table

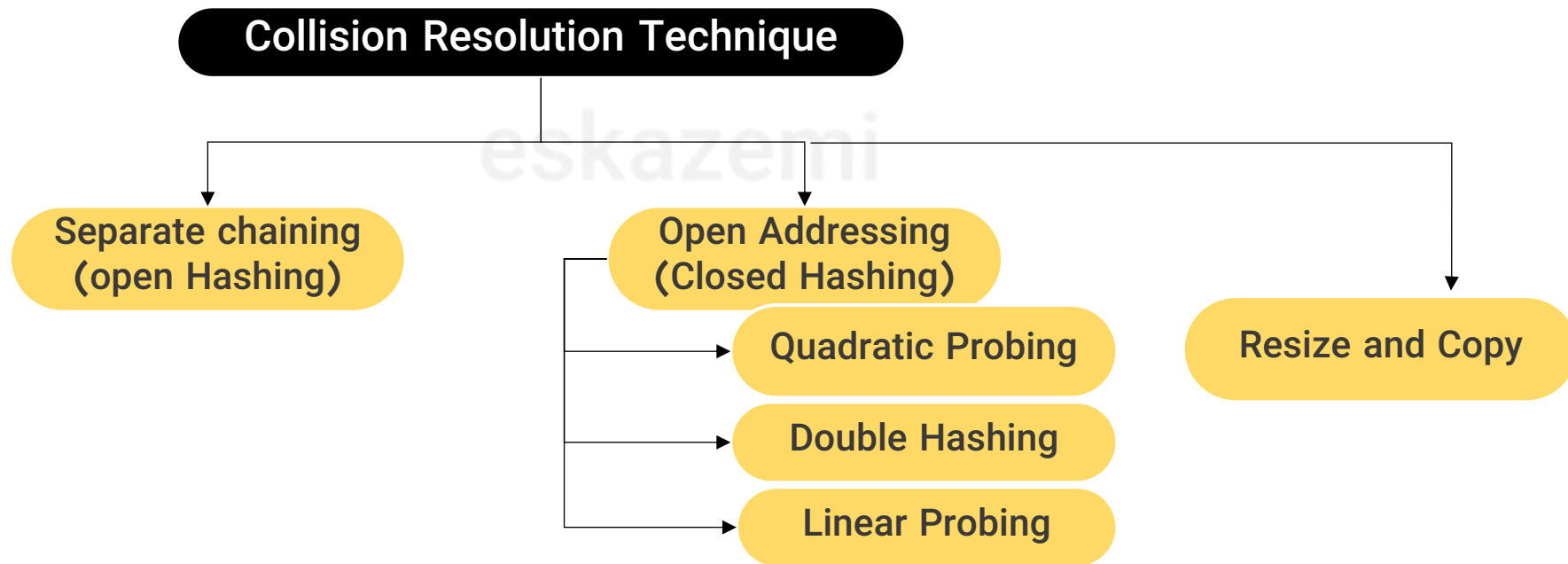
Collision in Hashing



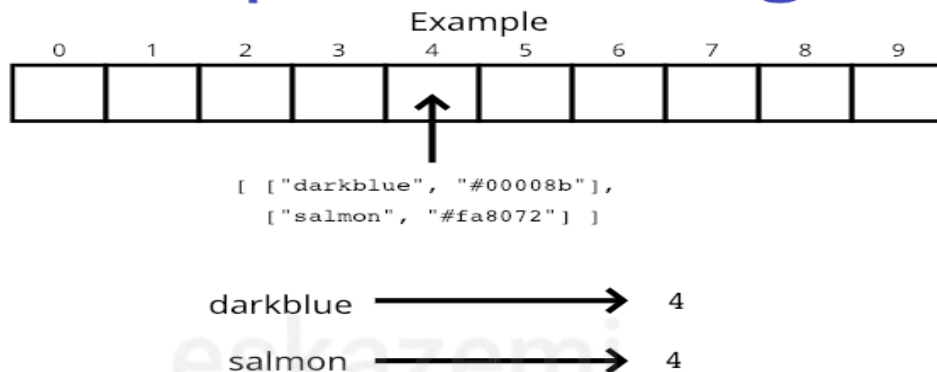
همان طور که پیش تر بررسی کردیم، توابع هش مجموعه ای نامتناهی از کلیدها (ورودی های با طول متغیر) را به مجموعه ای متناهی از اندیس ها (با طول ثابت) نگاشت می کنند. طبق اصول ریاضی، نگاشت یک مجموعه بی نهایت به یک فضای محدود، دیر یا زود منجر به وقوع تصادم خواهد شد.

در جداول هش، **تصادم (Collision)** به این معناست که تابع هش، چندین کلید مختلف را به یک اندیس یکسان - و در نتیجه به یک بلوک حافظه (Bucket) تکراری - هدایت کرده است.

راهکارهای مدیریت تصادم (Collision Resolution)

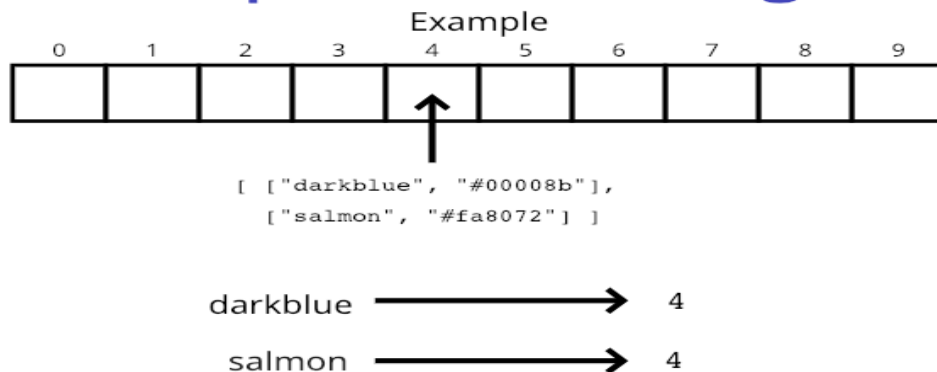


Separate Chaining



در راهبرد زنجیره‌سازی مجزا (Separate Chaining)، ما بر سر همان اندیسی که تصادم در آن رخ داده توافق می‌کنیم، اما از یک ساختار داده کمکی برای ذخیره‌سازی چند مقدار در آن اندیس واحد بهره می‌بریم. به عنوان مثال، به تصویری که در آن کدهای هگزادسیمال رنگ‌ها ذخیره شده‌اند دقت کنید: تابع هش برای کلید darkblue، اندیس شماره 4 را محاسبه می‌کند. سپس تابع هش برای کلید salmon نیز مجدداً همان اندیس 4 را تولید می‌کند (تصادم). ما به جای بازنویسی مقدار موجود در اندیس 4 و از دست دادن اطلاعات رنگ darkblue، هر دو جفت (کلید-مقدار) را درون یک آرایه (یا لیست پیوندی) نگهداری کرده و آن آرایه را به اندیس 4 متصل می‌کنیم.

Separate Chaining



نکات ظریف فنی در این روش:

- ✓ **تغییر در زمان اجرا (Time Complexity):** نقطه قوت **Hash Table**، دسترسی در زمان $O(1)$ است. اما در این روش، اگر تصادمها زیاد باشد و زنجیره‌ای طولانی در یک اندیس (مثلاً اندیس ۴) ایجاد شود، برای یافتن رنگ salmon سیستم باید ابتدا در زمان $O(1)$ به اندیس ۴ برود و سپس یک جستجوی خطی در بین عناصر آن زنجیره انجام دهد. در نتیجه، زمان دسترسی در بدترین حالت به $O(k)$ تنزل پیدا می‌کند (که در آن k طول آن زنجیره خاص است).
- ✓ **پویایی حافظه:** این تکنیک از نظر مصرف حافظه منعطف است. لیست‌های پیوندی تنها در صورت وقوع تصادم و در همان اندیس درگیر (به صورت پویا) ایجاد می‌شوند و نیازی به رزرو فضای خالی از پیش تعیین شده برای تصادمها نیست.

(Open Addressing / Closed Hashing)

آدرس‌دهی باز (Open Addressing / Closed Hashing)

برخلاف روش «زنجیره‌سازی مجزا» که در صورت وقوع تصادم از ساختارهای ثانویه (مثل لیست‌های پیوندی) کمک می‌گرفت، در خانواده آدرس‌دهی باز یک قانون سخت‌گیرانه وجود دارد: **تمام داده‌ها باید منحصرأ درون خانه‌های (Slots) خود آرایه اصلی ذخیره شوند.** هیچ لیست پیوندی یا فضای اضافه‌ای در خارج از جدول هش وجود ندارد.

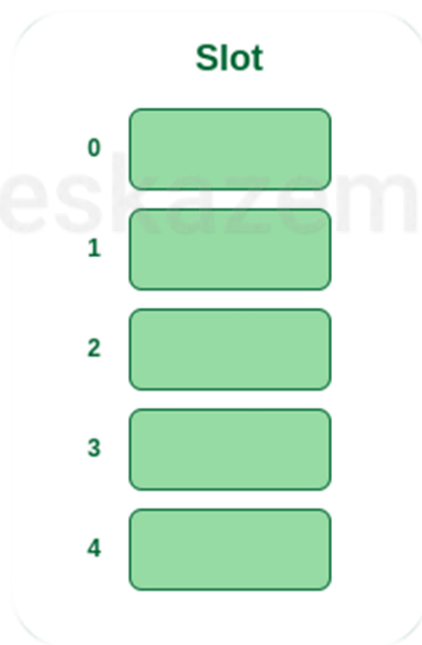
در این استراتژی، اگر تابع هش ما را به خانه‌ای هدایت کند که از قبل اشغال شده است، الگوریتم به جای توقف یا بازنویسی داده، یک فرآیند «پیمایش یا جستجو» (Probing) را در داخل همان آرایه آغاز می‌کند تا اولین خانه خالی را بیابد. پایه‌ای‌ترین و معروف‌ترین الگوریتم در این خانواده، **جستجوی خطی (Linear Probing)** است.

Linear Probing

مکانیزم جستجوی خطی (Linear Probing) چگونه کار می‌کند؟

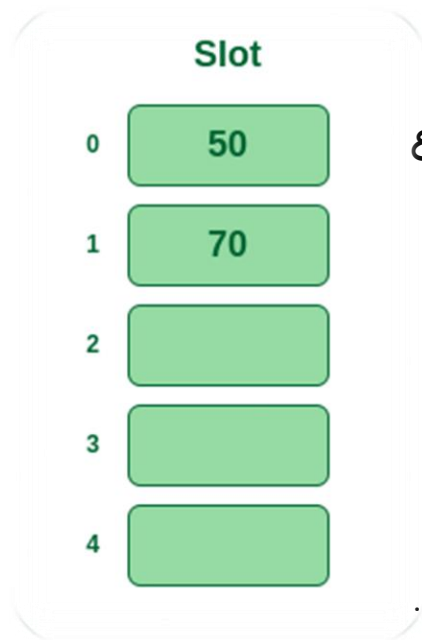
قاعده جستجوی خطی بسیار ساده است: «اگر خانه هدف اشغال بود، خانه‌های بعدی را یکی پس از دیگری (به صورت متوالی) بررسی کن تا به یک جای خالی برسی».

فرض کنید می‌خواهیم دنباله
کلیدهای [93, 76, 70, 50] را با
استفاده از تابع هش پایه $h(k)$
 $h(k) = k \pmod{5}$ در یک جدول
قرار دهیم:

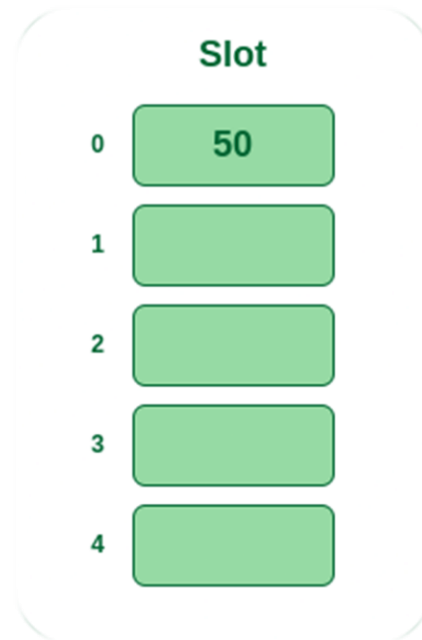


مرحله ۱ (آماده‌سازی): از آنجایی که تابع هش
ما بر مبنای پیمانه ۵ (Modulo 5) کار می‌کند،
خروجی‌ها همیشه عددی بین ۰ تا ۴ خواهند
بود. پس یک آرایه خالی با ۵ اسلات (از
اندیس ۰ تا ۴) ایجاد می‌کنیم.

Linear Probing



مرحله ۳ (درج کلید ۷۰ - وقوع اولین تصادم): محاسبه می‌کنیم $70 \bmod 5 = 0$: تابع هش مجدداً اسلات ۰ را پیشنهاد می‌دهد. اما این اسلات اکنون توسط عدد ۵۰ اشغال شده است! واکنش *Linear Probing*: الگوریتم یک گام به جلو می‌رود و اسلات بعدی یعنی شماره ۱ را بررسی می‌کند. چون اسلات ۱ خالی است، عدد ۷۰ در آنجا درج می‌شود.



مرحله ۲: حالا تمام کلیدها را در جدول هش یک به یک وارد کنید. کلید اول ۵۰ است. به دلیل اینکه $50 \bmod 5 = 0$ پس آن را در اسلات شماره ۰ وارد کنید.

Linear Probing

مرحله ۵: کلید
بعدی ۹۳ است که
به دلیل $93\%5=3$
به اسلات شماره ۳
منتقل می شود،
پس آن را در
اسلات شماره ۳
وارد کنید.

Slot	
0	50
1	70
2	76
3	93
4	

Slot	
0	50
1	70
2	76
3	
4	

کلید بعدی ۷۶ است.
برای اسلات شماره ۱
نگاشت می شود زیرا
 $76\%5=1$ اما ۷۰ در
حال حاضر در اسلات
شماره ۱ قرار دارد
بنابراین، اسلات خالی
بعدی را جستجو کرده
و آن را درج می کند.

(نکته: اگر بخواهیم در این جدول کلید ۸۵ را نیز درج کنیم، با توجه به $85 \pmod{5} = 0$ ، الگوریتم به اندیس ۰ می رود، سپس به دلیل پر بودن خانه ها، اندیس های ۱، ۲ و ۳ را یکی یکی طی می کند تا در نهایت در تنها خانه خالی یعنی اندیس ۴ آرام بگیرد).

گرچه پیاده‌سازی جستجوی خطی آسان است و به دلیل عدم استفاده از اشاره‌گرها (Pointers) **حافظه کمتری** نسبت به Chaining مصرف می‌کند، اما از نظر فنی با دو چالش اساسی روبرو است:

۱. مشکل تجمع اولیه (Primary Clustering):

همان‌طور که در مثال بالا دیدید، تصادم‌ها باعث می‌شوند داده‌ها به صورت بلوک‌های متصل به هم (خوشه) در آرایه جمع شوند. کلید 76 با اینکه هش اصلی‌اش 1 بود، مجبور شد در اندیس 2 بنشیند. حالا اگر کلیدی بیاید که هش آن واقعاً 2 باشد، باید به اندیس 3 یا 4 رانده شود! این اثر دومینووار باعث می‌شود خوشه‌های بزرگی از داده در کنار هم شکل بگیرد که زمان جستجو (Search) و درج (Insert) را به شدت کند کرده و از $O(1)$ به سمت $O(n)$ سوق می‌دهد.

۲. چرخش مدور (Circular Probing):

سوال مهمی که در پیاده‌سازی پیش می‌آید این است: «اگر در انتهای آرایه تصادم رخ داد و خانه بعدی وجود نداشت چه؟». در آدرس‌دهی باز، آرایه به صورت یک حلقه (Circular) فرض می‌شود. یعنی اگر جستجو به آخرین اندیس (مثلاً اندیس 4) رسید و آنجا پر بود، الگوریتم به جای توقف (Overflow)، دوباره به ابتدای آرایه یعنی اندیس 0 بازمی‌گردد تا خانه‌های خالی جا مانده را پیدا کند. از نظر ریاضی، فرمول دقیق جستجوی خطی در مرحله i ام به شکل زیر است:

$$P(k, i) = (h(k) + i) \pmod{N}$$

(که در آن N ظرفیت کل جدول است).

۳. محدودیت مطلق فضا:

برخلاف روش زنجیره‌سازی که می‌توانست تا بی‌نهایت داده را در خود جای دهد (فضای پویا)، در آدرس‌دهی باز ظرفیت جدول دقیقاً برابر با سایز آرایه است. اگر ۵ اسلات داریم، فقط ۵ داده می‌توانیم ذخیره کنیم. زمانی که جدول در حال پر شدن است (افزایش Load Factor)، تعداد تصادم‌ها به صورت نمایی بالا می‌رود و سیستم موظف است عملیات گران‌قیمت **Resize and Copy** (بزرگ کردن سایز آرایه و بازتوزیع مجدد تمام هش‌ها) را انجام دهد.

فرار از ترافیک: راهکارهای پیشرفته در آدرس‌دهی باز
همان‌طور که دیدیم، مشکل اصلی در روش «جستجوی خطی» این بود که وقتی تصادمی رخ می‌داد، داده‌ها دقیقاً در خانه همسایه پارک می‌کردند. این رفتار باعث ایجاد «ترافیک سنگین» (Primary Clustering) در بخش‌هایی از حافظه می‌شد. مهندسان نرم‌افزار برای حل این مشکل، دو الگوریتم هوشمندانه‌تر طراحی کردند که به جای قدم‌زدن یکی‌یکی، با «پرش‌های بلند» به دنبال خانه خالی می‌گردند.

۱. جستجوی توان‌دار (Quadratic Probing)

در این روش، وقتی تصادمی رخ می‌دهد، الگوریتم به جای اینکه خانه‌های بعدی را به ترتیب $1, 2, 3, \dots$ چک کند، فاصله جستجو را به صورت **توانی** $(1^2, 2^2, 3^2, \dots)$ افزایش می‌دهد.
فرمول ریاضی:

$$Index(i) = (h(k) + i^2) \pmod{N}$$

(که در آن $h(k)$ هاش اولیه، i شماره تلاش، و N سایز جدول است)
مثال عملی:

فرض کنید جدول هشی با سایز $N = 10$ داریم و کلید A قبلاً در خانه شماره ۲ نشسته است. حالا می‌خواهیم کلید B را درج کنیم که هاش آن هم دقیقاً ۲ می‌شود (تصادم!).

✓ **تلاش اول** ($i = 0$): خانه شماره ۲ پر است.

✓ **تلاش دوم** ($i = 1$): به جای رفتن به خانه ۳ (مثل روش خطی)، فرمول می‌گوید: $(2 + 1^2) = 3$. اگر خانه ۳ هم پر باشد...

✓ **تلاش سوم** ($i = 2$): اینجا تفاوت اصلی دیده می‌شود. به جای خانه ۴، الگوریتم می‌پرد به: $(2 + 2^2) = 6$.

✓ **تلاش چهارم** ($i = 3$): پرش بعدی به فاصله ۹ خانه است: $1 \rightarrow (2 + 3^2) = 11$ (با پیمانه ۱۰).

مزیت فنی: این پرش‌های بلند باعث می‌شود داده‌ها کنار هم تلنبار نشوند و در کل سطح جدول پخش شوند. این کار مشکل «خوشه‌بندی اولیه» را تا حد زیادی حل می‌کند.

عیب فنی (Secondary Clustering): اگر دو کلید مختلف دقیقاً به یک هاش اولیه برسند (مثلاً هر دو هاش‌شان ۲ باشد)، دقیقاً همان مسیر پرش‌های تکراری $(2 \rightarrow 3 \rightarrow 6 \rightarrow \dots)$ را طی می‌کنند. به این پدیده «خوشه‌بندی ثانویه» می‌گویند.

۲. هشینگ مضاعف (Double Hashing)

این تکنیک، قوی‌ترین و کارآمدترین روش در بین روش‌های آدرس‌دهی باز است. ایده اصلی این است: «چرا برای پرش‌ها از یک الگوریتم ثابت استفاده کنیم؟ بیایید طول پرش را هم با یک تابع هش دیگر محاسبه کنیم!» در اینجا ما دو تابع هش داریم:

1. $h_1(k)$: آدرس اولیه را مشخص می‌کند.

2. $h_2(k)$: «طول گام‌های پرش» را در صورت بروز تصادم تعیین می‌کند.

فرمول ریاضی:

$$Index(i) = (h_1(k) + i \times h_2(k)) \pmod{N}$$

فرض کنید جدول سایز 10 دارد.

$$\bullet h_1(k) = k \pmod{10}$$

$$\bullet h_2(k) = 7 - (k \pmod{7}) \quad (\text{یک تابع هاش دوم فرضی})$$

حالا می‌خواهیم کلید $k = 82$ را درج کنیم، اما خانه شماره 2 (که خروجی h_1 است) پر شده.

$$1. \text{محاسبه گام پرش: } h_2(82) = 7 - (82 \pmod{7}) = 7 - 5 = 2.$$

پس الگوریتم می‌فهمد که برای این کلید خاص، باید 2 تا 2 تا بپرد.

$$2. \text{تلاش بعدی: } (2 + 1 \times 2) = 4. \text{ اگر پر بود...}$$

$$3. \text{تلاش بعدی: } (2 + 2 \times 2) = 6.$$

حالا فرض کنید کلید دیگری مثل $k = 92$ بیاید که h_1 آن هم 2 است (تصادم با کلید قبلی).

$$1. \text{محاسبه گام پرش برای } 92: h_2(92) = 7 - (92 \pmod{7}) = 7 - 1 = 6.$$

برای این کلید، سیستم تصمیم می‌گیرد 6 تا 6 تا بپرد.

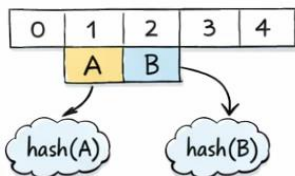
$$2. \text{تلاش بعدی: } (2 + 1 \times 6) = 8.$$

تحلیل نهایی:

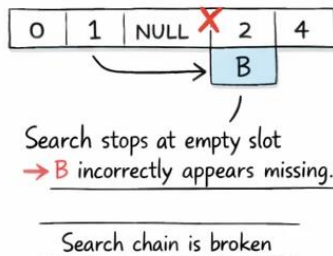
دقت کردید؟ با اینکه هر دو کلید 82 و 92 در خانه شماره 2 تصادم کردند و سرنوشت یکسانی داشتند، اما مسیر بعدی‌شان کاملاً متفاوت شد (یکی به خانه 4 رفت و دیگری به خانه 8). این ویژگی باعث می‌شود عملاً پدیده خوشه‌بندی (Clustering) به طور کامل حذف شود و توزیع داده‌ها در حافظه به حالت ایده‌آل (Uniform Hashing) بسیار نزدیک شود.

Deletion Problem in Open Addressing (Linear Probing)

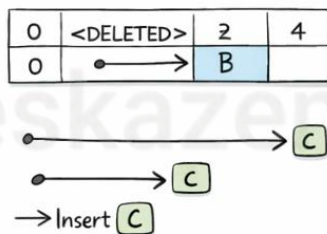
Before Deletion



Wrong Deletion (Setting NULL)



Correct Engineering Solution – Tombstone



چالش اساسی: حذف
داده در آدرس دهی باز
(Tombstone)

In open addressing, deleted entries are marked with a Tombstone so search chains remain intact.

In separate chaining, deletion is simple because nodes are removed from a linked list.

معمای حذف داده‌ها (Deletion Problem) و تکنیک سنگ قبر (Tombstone)

حذف کردن یک کلید در روش‌های آدرس‌دهی باز (Open Addressing) مانند جستجوی خطی (Linear Probing)، یکی از خطرناک‌ترین عملیات‌هاست و اگر درست هندل نشود، کل جدول از کار می‌افتد!

• صورت مسئله (چرا حذف ساده خطرناک است؟):

فرض کنید با الگوریتم جستجوی خطی، کلید A در اندیس ۱، و کلید B (که با A تصادم داشته) در اندیس ۲ ذخیره شده است. اگر ما کلید A را پاک کنیم و اندیس ۱ را کاملاً «خالی» (Null) کنیم، چه می‌شود؟
دقیقه بعد، سیستم می‌خواهد کلید B را پیدا کند. تابع هاش به اندیس ۱ اشاره می‌کند. سیستم می‌بیند اندیس ۱ «خالی» است. بر اساس منطق جستجوی خطی، سیستم با خود می‌گوید: «چون به خانه خالی رسیدم، پس حتماً کلید B اصلاً در این جدول وجود ندارد!» و عملیات جستجو را متوقف می‌کند؛ در حالی که B زنده بود و در اندیس ۲ قرار داشت! (به این حالت قطع شدن زنجیره جستجو می‌گویند).

• راهکار مهندسی (Tombstone یا Dummy Flag):

مهندسان نرم‌افزار برای حل این مشکل، هرگز خانه‌ها را واقعاً پاک یا Null نمی‌کنند. بلکه وقتی داده‌ای حذف می‌شود، در آن خانه یک علامت ویژه قرار می‌دهند که به آن **سنگ قبر (Tombstone)** یا نشانگر <DELETED> می‌گویند.

✓ **هنگام جستجو (Search):** اگر الگوریتم به یک خانه Tombstone برسد، می‌فهمد که اینجا قبلاً پر بوده و حذف شده است، پس زنجیره قطع نشده و باید به جستجوی خود در خانه‌های بعدی ادامه دهد.

✓ **هنگام درج جدید (Insert):** اگر الگوریتم بخواهد داده جدیدی وارد کند و به Tombstone برسد، با خیال راحت داده جدید را روی آن می‌نویسد (چون آن داده قبلاً مرده است) و از فضای حافظه استفاده بهینه می‌کند.

(نکته: در روش *Separate Chaining* که از لیست پیوندی استفاده می‌کند، این مشکل وجود ندارد و خیلی راحت گره (Node) از لیست پیوندی حذف می‌شود).

«کالبدشکافی دیکشنری پایتون در پشت صحنه»

چرا پایتون علاوه بر Key و Value، خود مقدار هش (Hash) را هم ذخیره می‌کند؟

در پیاده‌سازی دیکشنری‌های پایتون (و همچنین Set ها)، هر ردیف از جدول هش فقط شامل دو ستون (کلید و مقدار) نیست، بلکه سه بخش دارد: [Hash, Key, Value]. پایتون با این کار مقداری از حافظه (Space) را فدا می‌کند تا سرعت (Time) را به شدت بالا ببرد (همان مفهوم موازنه فضا-زمان). دلیل این کار دو معماری بسیار مهم است:

الف) افزایش سرعت مقایسه (Fast Comparison) هنگام وقوع تصادم:

وقتی دو کلید در یک اندیس تصادم می‌کنند، پایتون باید چک کند که آیا این همان کلیدی است که کاربر دنبالش می‌گردد یا خیر. مقایسه کردن دو رشته طولانی (String) کار زمان‌بری است. پایتون به جای مقایسه رشته‌ها، ابتدا مقدار هش ذخیره‌شده آن‌ها را (که یک عدد صحیح یا Integer است) با هم مقایسه می‌کند. مقایسه اعداد در پردازنده (CPU) در یک کلاک‌سایکل انجام می‌شود! اگر هش‌ها برابر نبودند، پایتون مطمئن می‌شود که کلیدها هم قطعاً متفاوتند و اصلاً سراغ مقایسه رشته‌ها نمی‌رود. تنها زمانی رشته‌ها را حرف‌به‌حرف مقایسه می‌کند که هش‌ها دقیقاً یکسان باشند.

ب) سرعت در تغییر سائز جدول (Fast Rehashing):

همان‌طور که در سوال دوم اشاره کردید، جدول‌ها گاهی بزرگ می‌شوند. وقتی جدول بزرگ می‌شود، باید جای تمام داده‌ها دوباره محاسبه شود (Rehashing). اگر پایتون مقدار هش را ذخیره نکرده بود، مجبور بود برای تک‌تک میلیون‌ها کلید موجود در دیکشنری، دوباره تابع هش را اجرا و محاسبه کند که فاجعه پرفورمنس بود. با داشتن مقدار هش آماده، فقط یک عملیات ساده ریاضی (مثلاً $(Hash \bmod New_Size)$) انجام می‌دهد و داده‌ها را به خانه جدید منتقل می‌کند.

«مدیریت حافظه و استراتژی تغییر سایز (Resizing) در جداول هش»

ضریب بار (Load Factor) و عملیات تغییر سایز (Resizing)

جدول هش نمی‌تواند تا بی‌نهایت پر شود. یک مفهوم ریاضی بسیار مهم در اینجا وجود دارد به نام **ضریب بار (Load Factor)** که با نماد α نشان داده می‌شود:

$$\alpha = \frac{n}{m}$$

(که در آن n تعداد آیتم‌های ذخیره شده و m ظرفیت کل آرایه است).

هرچه α به عدد 1 (یعنی پر شدن ۱۰۰ درصدی) نزدیک شود، احتمال تصادم‌ها (Collisions) به شدت بالا می‌رود و به جای جستجوی $O(1)$ ، الگوریتم مجبور است خانه‌های زیادی را بگردد و سرعت به سمت $O(n)$ سقوط می‌کند.

✓ **راهکار مهندسی (Dynamic Resizing):**

در مفسر پایتون (CPython)، یک قانون سخت‌گیرانه وجود دارد: «دیکشنری هرگز نباید بیشتر از ۶۶ درصد (دو سوم) پر شود!». یعنی وقتی ضریب بار به $\alpha \approx 0.66$ می‌رسد، پایتون زنگ خطر را به صدا در می‌آورد.

✓ **فرآیند Resize:** پایتون در پس‌زمینه یک آرایه جدید با **دو برابر** ظرفیت قبلی می‌سازد. سپس تمام داده‌های جدول قدیمی را برداشته و با استفاده از همان **هشی** که ذخیره کرده بود (پاسخ سوال اول)، آن‌ها را در آرایه جدید پخش می‌کند. این عملیات کپی کردن مقداری زمان‌بر است ($O(n)$)، اما چون خیلی به ندرت اتفاق می‌افتد، در میانگین زمانی (Amortized Time) همچنان عملکرد دیکشنری $O(1)$ باقی می‌ماند.

قلب تپنده سیستم: کالبدشکافی تابع هش (Hash Function)

تابع هش هسته مرکزی تمام عملیات در یک جدول هش است. اگر آرایه را کالبد فیزیکی جدول بدانیم، تابع هش مغز متفکر آن است که تصمیم می‌گیرد هر داده کجا بنشیند. اما یک تابع ریاضی برای اینکه لیاقت مدیریت داده‌ها را پیدا کند، باید استانداردهای مهندسی سخت‌گیرانه‌ای را برآورده سازد.

۱. ویژگی‌های ذاتی یک تابع هش ایده‌آل (The Must-Haves) محاسبه‌پذیری بی‌درنگ (Efficiency):

یک تابع هش باید بسیار سبک و سریع باشد.

✓ پیچیدگی زمانی: زمان محاسبه هش به طور کلی $O(k)$ است (که در آن k طول کلید است، مثلاً طول یک رشته متنی)، اما در مقیاس تعداد کل داده‌های جدول (N)، این زمان ثابت است و در نتیجه $O(1)$ در نظر گرفته می‌شود.

✓ پیچیدگی فضایی: برای تولید خروجی، تابع نباید حافظه اضافی اشغال کند، بنابراین حافظه مصرفی آن کاملاً $O(1)$ است.

قطعیت صددرصدی (Determinism):

قانون طلایی هشینگ: «تا زمانی که ورودی تغییر نکند، خروجی باید بی‌نهایت بار یکسان بماند». اگر کلید "123User" امروز به اندیس 5 ختم می‌شود، فردا هم باید خروجی آن 5 باشد. در غیر این صورت، داده‌ها در حافظه گم می‌شوند.

توزیع کاملاً یکنواخت (Uniform Distribution):

تابع هش نباید به بخش خاصی از آرایه علاقه نشان دهد. شانس قرار گرفتن یک داده در اندیس 0 باید دقیقاً برابر با شانس قرار گرفتن آن در اندیس 99 باشد. توزیع نابرابر، مستقیماً به تصادم (Collision) و افت پرفورمنس منجر می‌شود.

اندازه خروجی ثابت (Fixed-Length Output / Compression):

شما می‌توانید یک کلمه ۵ حرفی یا متن یک کتاب ۵۰۰ صفحه‌ای را به عنوان کلید ورودی بدهید؛ تابع هش همیشه یک خروجی با طول ثابت (مثلاً یک عدد صحیح ۳۲ بیتی) تولید می‌کند. به همین دلیل در علوم کامپیوتر به توابع هش، فشرده‌ساز (Compressor) یا چکیده‌ساز (Digest) نیز می‌گویند.

مرزهای مهندسی: هشینگ ساختار داده vs هشینگ رمزنگاری

یکی از اشتباهات رایج، یکی دانستن توابع هش جداول با توابع هش امنیتی است. در مهندسی نرم افزار این دو کاربرد و معماری کاملاً متفاوتی دارند:

الف) هش‌های غیررمزنگاری (Non-Cryptographic - مناسب برای Hash Tables):

هدف در اینجا فقط «سرعت» و «پراکندگی خوب» است. نیازی نیست یک طرفه باشند. به عنوان مثال، توابعی مثل MurmurHash یا CityHash (توسعه یافته توسط گوگل) در جداول هش و دیتابیس‌ها استفاده می‌شوند چون در کسری از نانو ثانیه اجرا می‌شوند.

ب) هش‌های رمزنگاری (Cryptographic - مناسب برای امنیت و پسورد):

هدف در اینجا «امنیت مطلق» و «یک طرفه بودن (One-Way)» است. الگوریتم‌هایی مثل SHA-256 یا bcrypt. در اینجا تبدیل ورودی به خروجی امکان پذیر است، اما مهندسی معکوس خروجی برای رسیدن به ورودی عملاً **غیرممکن** است (حتی با ابر کامپیوترها). این توابع عمداً **کند** طراحی می‌شوند تا در برابر حملات Brute-force مقاوم باشند. هیچ مهندس نرم افزاری از SHA-256 برای پیاده سازی یک Dictionary ساده در پایتون استفاده نمی‌کند، چون سیستم به شدت **کند** می‌شود!

الگوریتم‌های کلاسیک محاسبه مقدار هش

برای تبدیل یک کلید (چه عددی و چه رشته‌ای) به یک آدرس معتبر در حافظه، ریاضیات مختلفی در پس‌زمینه وجود دارد. رایج‌ترین متدها عبارتند از:

1. روش تقسیم (Division Method):

ساده‌ترین روش که با فرمول $h(k) = k \pmod{M}$ کار می‌کند (M سایز جدول است). نکته مهندسی: برای حداقل رساندن تصادم در این روش، مهندسان همیشه سایز جدول (M) را یک عدد اول (Prime Number) انتخاب می‌کنند که به توان‌های ۲ نزدیک نباشد.

2. روش ضرب (Multiplication Method):

در این روش، کلید در یک عدد اعشاری ثابت (بین صفر و یک) ضرب می‌شود، بخش اعشاری آن جدا شده و در سایز جدول ضرب می‌گردد: $h(k) = \lfloor M \times ((k \times A) \pmod{1}) \rfloor$. مزیت این روش این است که برخلاف روش تقسیم، وابستگی شدیدی به سایز دقیق جدول (M) ندارد.

3. روش مربع میانی (Mid Square Method):

کلید را به توان دو می‌رساند و ارقام وسط آن را به عنوان آدرس استخراج می‌کند. مثلاً اگر کلید 12 باشد، مربع آن 144 است و رقم وسط یعنی 4 به عنوان آدرس انتخاب می‌شود. این روش پراکندگی بسیار خوبی ایجاد می‌کند.

4. روش تا کردن (Folding Method):

بیشتر برای کلیدهای طولانی (مثل شماره تلفن) استفاده می‌شود. کلید به بخش‌های مساوی تقسیم شده و آن بخش‌ها با هم جمع می‌شوند. مثلاً کلید 4357 به دو بخش 43 و 57 تقسیم شده و آدرس برابر با $43 + 57 = 100$ خواهد بود.

نگاه از لنز DevOps و معماری سیستم

استفاده از توابع هش فقط محدود به برنامه‌نویسی سطح پایین نیست. در زیرساخت و معماری کلان نرم‌افزار، هشینگ نقش حیاتی دارد:

✓ توزیع بار (Load Balancing) و هشینگ یکنواخت (Consistent Hashing):

وقتی در یک سیستم ابری میلیون‌ها درخواست به سمت سرورها می‌آید، ابزارهای Load Balancer (مثل NGINX) از آدرس IP کاربر یک هش تولید می‌کنند. این هش تعیین می‌کند که درخواست کاربر باید دقیقاً به کدام سرور فیزیکی (مثلاً سرور A یا B) هدایت شود تا بار سیستم به طور مساوی تقسیم گردد.

✓ کشینگ (Caching) در ابزارهایی مثل Redis:

دیتابیس‌های In-Memory مثل Redis در واقع یک Hash Table بسیار غول‌پیکر هستند که در RAM سرور اجرا می‌شوند. جستجوی $O(1)$ باعث می‌شود اپلیکیشن‌ها داده‌های پرکاربرد را در میکروثانیه فراخوانی کنند.

✓ یکپارچگی کانتینرها (Docker) و فایل‌ها:

در دنیای DevOps، هر لایه از یک Image داکر یا هر فایل در گیت (Git) دارای یک شناسه هش منحصر به فرد (معمولاً SHA) است. اگر حتی یک کاراکتر در فایل تغییر کند، تابع هش خروجی کاملاً متفاوتی (Avalanche Effect) تولید می‌کند و سیستم CI/CD متوجه تغییر شده و از استقرار کد خراب جلوگیری می‌کند.

کاربردهای فراتر هشینگ در امنیت و شبکه

کاربردهای گسترده‌تر هشینگ

اگرچه جداول هش مهم‌ترین کاربرد توابع هش در زمینه ساختار داده هستند، اما رسالت هشینگ به اینجا ختم نمی‌شود. به دلیل خاصیت «خروجی ثابت و یک‌طرفه بودن» (در توابع هش رمزنگاری)، از این مکانیزم در حوزه‌های حیاتی دیگری نیز استفاده می‌شود:

- ❖ ذخیره‌سازی و اعتبارسنجی رمزهای عبور (Passwords) به صورت امن
- ❖ ایجاد امضای دیجیتال (Message Signatures)
- ❖ بررسی یکپارچگی فایل‌ها (Data Integrity Verification)

Thanks!

Any questions?

You can find me at:

- @eskazemi
- m.esmaeilkazemi@gmail.com

